TOPIC - 8

Arduino I2C: The Wire Library, Master & Slave Roles

INSTRUCTOR: ALIJAVAD SADIKHZADA

BAKU HIGHER OIL SCHOOL

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- **I2C** (Inter-Integrated Circuit) — also called **IIC** or **TWI** (Two-Wire Interface) — needs only two wires no matter how many devices share the bus, each identified by its own address rather than its own dedicated pin.
- **SDA** (data) and **SCL** (clock) were already named as I2C's two pins; what follows is how those two wires actually carry a full conversation between a master and any number of devices.
- Where SPI needs a separate Slave Select pin for every device, and UART only ever connects two points directly, I2C's addressing is what lets dozens of sensors and modules share the exact same two wires.

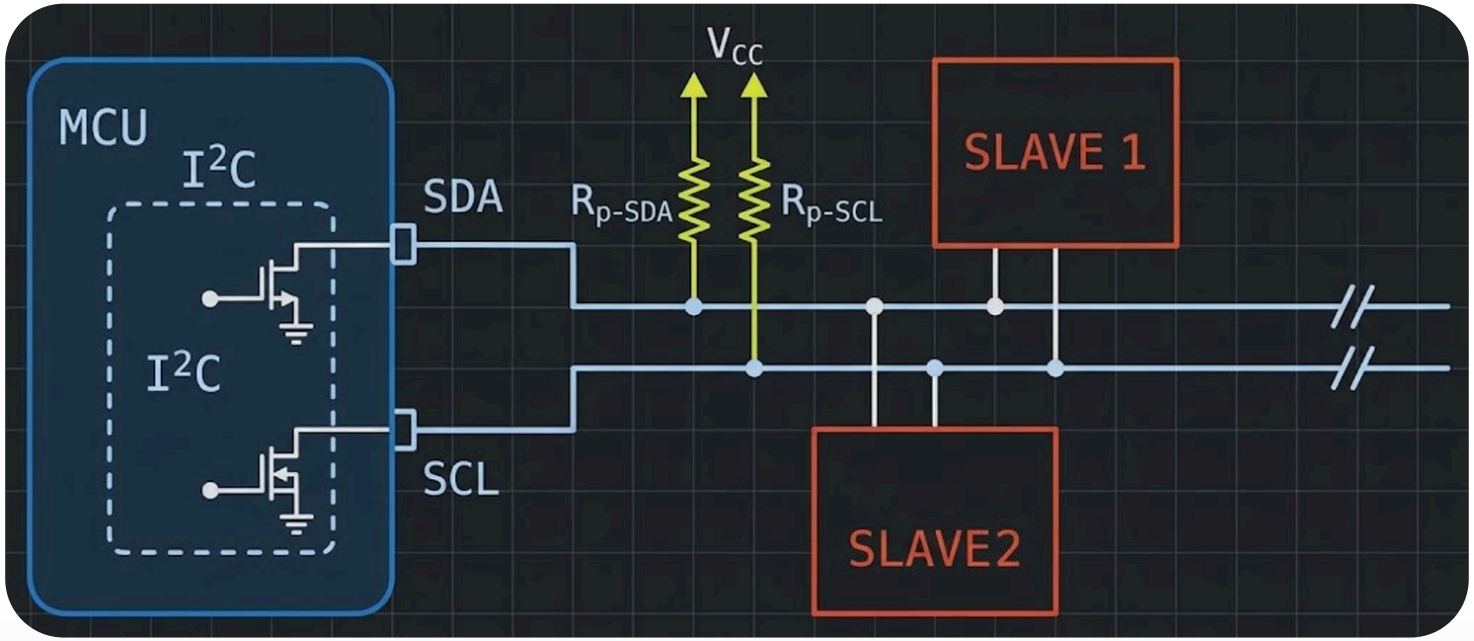
RTC clocks, OLED displays, I/O expanders, temperature and humidity sensors, EEPROMs, and the MPU6050 IMU are all typically wired this way — two wires, however many of them are on the bus at once.

Why I2C?

- Both SDA and SCL are **open-drain** lines — no device can drive a line high, only pull it low; a pull-up resistor is what holds each line high whenever nothing is pulling it down.
- Because a 0 always overrides a 1 on an open-drain line, two devices can safely drive the bus at the same moment — whichever one is pulling low wins, with nothing damaged on either side.
- The Wire library turns on weak internal pull-ups (roughly 20 to 50 kΩ) by default; external resistors, 4.7 kΩ or 2.2 kΩ, are the more reliable choice and are what most datasheets assume.

This is also what makes mixed voltages workable — a 3.3-volt-only device can sit on the same bus as 5-volt logic simply by pulling its own line up to 3.3 volts instead of 5, since the bus is never driven high, only released.

Open-Drain Lines and Pull-Ups



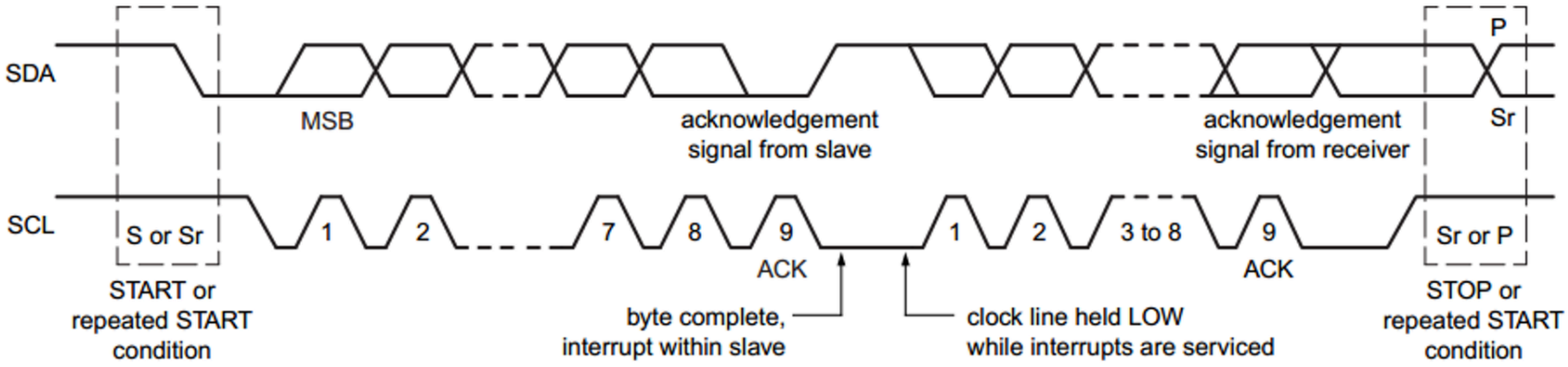
Each line's transistor can only pull its line down to ground — neither one can drive its line high. Rp-SDA and Rp-SCL are what pull each line back up to Vcc the instant every transistor on the bus releases it.

Open-Drain, Visualized

- A **start condition (S)** opens a transaction and a **stop condition (P)** closes it; a **repeated start (Sr)** switches direction mid-transaction without releasing the bus in between.
- Every transaction opens with an address — 7-bit by default, or 10-bit for a larger address space — immediately followed by one bit saying whether this is a read or a write.
- Every single byte, address included, gets one **ACK (acknowledge)** or **NACK (not-acknowledge)** bit back from whichever side just received it, confirming byte-by-byte whether the other side is still there and listening.

This is the structure every Wire library call is quietly building on top of — beginTransmission() and endTransmission() mark the start and stop conditions, and a failed acknowledgment is exactly what an endTransmission() error code is reporting back.

A Transaction, Piece by Piece



Data transfer on the I²C-bus

SCL ticks off eight data bits and a ninth ACK bit, byte after byte, between one start condition and one stop — the exact same eight-bits-plus-ACK pattern repeats for the address byte and for every data byte that follows it.

A Full Transaction, Visualized

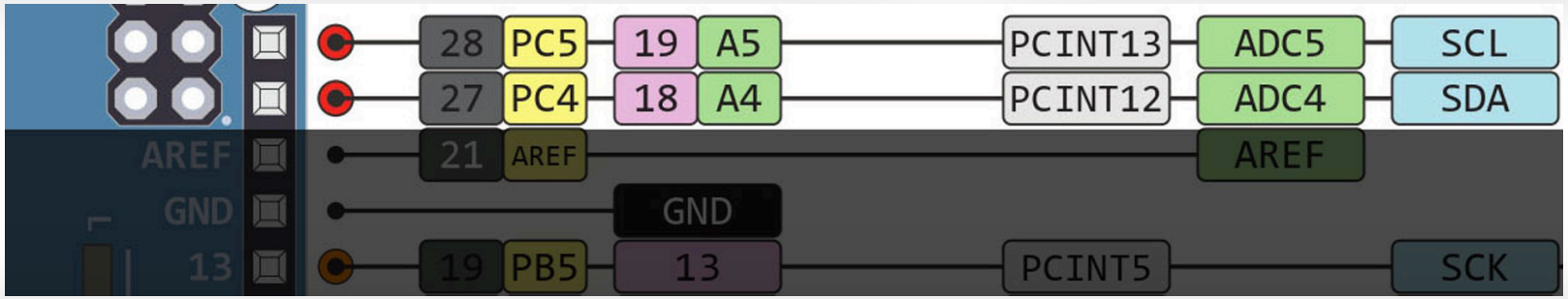
- **Standard mode** reaches 100 kbit/s, **Fast mode** reaches 400 kbit/s, **Fast-Mode Plus** reaches 1 Mbit/s, **High-speed mode** reaches 3.4 Mbit/s, and **Ultra Fast-mode** reaches 5 Mbit/s in one direction only.
- The ATmega328P tops out at Fast mode — the higher speeds exist for devices this board simply cannot reach.
- The Wire library defaults to 100 kHz regardless, and Wire.setClock() is what raises it, most commonly to 400000 for Fast mode.

A faster clock does not change anything about the transaction structure covered above — the same start/address/data/ACK sequence happens, just packed into less time per bit.

Speed Modes

- **Uno, Nano,** and **Pro Mini** — A4 (SDA) and A5 (SCL); these two pins cannot be used for analog input while I2C is active.
- **Mega** — pins 20 (SDA) and 21 (SCL).
- **Leonardo** and **Micro** — pins 2 (SDA) and 3 (SCL).

Losing A4 and A5 as analog inputs is easy to miss when a sketch is already using analogRead() elsewhere — on an Uno, any I2C device on the bus quietly claims both pins the moment Wire.begin() runs.



A4 and A5 are the same physical pins labeled PC4 and PC5 on the ATmega328P itself — SDA and SCL are not separate hardware, just a second job these two analog pins already had.

I2C Pins by Board

Wire: Master-Side Functions

- **Wire.begin()** joins the bus as master; **Wire.setClock(frequency)** sets the bus speed, typically 100000 or 400000.
- **Wire.beginTransmission(address)** opens a message to one device; **Wire.write(...)** queues a byte, a string, or an array of bytes into it; **Wire.endTransmission()** actually sends everything queued and returns a result code — 0 for success, anything else naming a specific failure.
- **Wire.requestFrom(address, quantity)** asks a device for bytes back; **Wire.available()** reports how many have arrived so far, and **Wire.read()** returns them one at a time.

All communication goes through a single shared 32-byte buffer, and only 7-bit addresses are used by default — addresses 0 through 7 are reserved and never assigned to a real device.

- **Wire.begin(address)** — the same function as the master-side join, just called with an address this time, which is what puts the board into slave mode instead.
- **Wire.onReceive(handler)** registers a function that runs automatically whenever a master sends this device data — the handler receives how many bytes arrived.
- **Wire.onRequest(handler)** registers a function that runs automatically whenever a master asks this device for data — it takes no parameters, and whatever it Wire.write()s becomes the reply.

Both callbacks run the same way an interrupt service routine does — triggered by hardware activity on the bus, not called directly by loop() — so the same ground rules already covered apply: keep them short, and avoid anything that blocks.

Wire: Slave-Side Callbacks

- A potentiometer reading is scaled down from 0-1023 to a single byte before being written — I2C bytes are 8 bits, exactly like Wire.write() expects.
- beginTransmission()/write()/endTransmission() is one complete send; requestFrom() immediately after is a separate, second transaction asking the same device for something back.
- Both transactions use the same fixed slave address, 0x55 here, since nothing on the bus distinguishes a send from a request except which function starts it.

```ino
#include <Wire.h>

const byte SLAVE_ADDR = 0x55;

void setup()
{
  Wire.begin();           // join the bus as master
  Wire.setClock(100000);  // 100 kHz, standard mode
  Serial.begin(9600);
}

void loop()
{
  int reading = analogRead(A0) >> 2;  // scale down to one byte

  Wire.beginTransmission(SLAVE_ADDR);
  Wire.write(reading);
  Wire.endTransmission();

  Wire.requestFrom(SLAVE_ADDR, 1);
  while (Wire.available()) {
    byte reply = Wire.read();
    Serial.println(reply);
  }

  delay(200);
}
```

Code: I2C Master

Code: I2C Slave

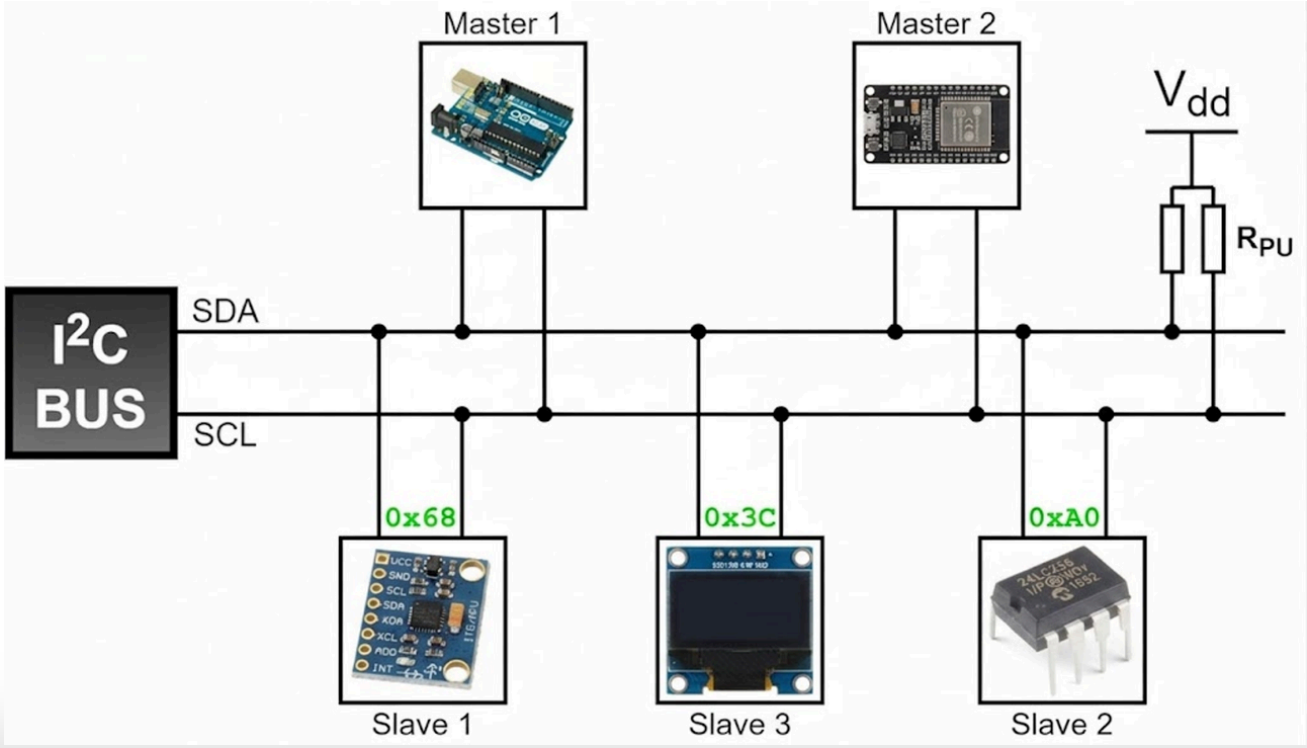
- Wire.begin(0x55) is the only line that differs from a master sketch — everything else is the two callbacks doing the actual work.
- onI2CReceive() stores whatever byte just arrived; onI2CRequest() answers with whatever was stored last, so the slave always has a sensible reply ready even if nothing new has come in since the last request.
- loop() does nothing at all here — the bus handles both callbacks on its own schedule, same as an ISR firing independently of whatever loop() happens to be doing.

```ino
#include <Wire.h>

volatile byte lastReading = 0;

void setup()
{
  Wire.begin(0x55);  // join the bus at this fixed address
  Wire.onReceive(onI2CReceive);
  Wire.onRequest(onI2CRequest);
  Serial.begin(9600);
}

void onI2CReceive(int howMany)
{
  while (Wire.available()) {
    lastReading = Wire.read();
  }
  Serial.println(lastReading);
}

void onI2CRequest()
{
  Wire.write(lastReading);
}

void loop()
{
  // nothing to do here
}
```

- **Single-master, single-slave** — the simplest case, one board talking to one device, such as reading a single MPU6050.
- **Single-master, multi-slave** — the most common real setup, one board sharing its two wires across several devices at once, an IMU and an EEPROM for example, each at its own address.
- **Multi-master, single-slave or multi-master, multi-slave** — more than one microcontroller on the same bus, relying on the open-drain arbitration already covered to avoid two masters colliding.

Every one of these is still just the same transaction structure repeated — more devices on the bus changes how many addresses exist, not how any single conversation between two of them works.

Bus Configurations



Two masters and three slaves share one SDA/SCL pair here — every device still answers to its own address (0x68, 0x3C, 0xA0), and the open-drain bus is what lets either master attempt to speak without damaging the other.

Multi-Master, Multi-Slave, Visualized

- **Master transmits, slave receives** — the pattern in the slave code above, one board pushing data to another whenever it wants to.
- **Master receives, slave transmits** — the pattern in the master code above, one board pulling data out of another on demand.
- **Both directions at once** — combine onReceive() and onRequest() on the slave with both write() and requestFrom() on the master, and the same two boards exchange data either way, on the same two wires.

This is the same two-board idea already built with fixed TX/RX wiring, now done through addresses instead of dedicated pins — the master decides when each direction happens, and the slave only ever reacts.

Two Boards, Three Patterns

- Two identical modules on the same bus is a real problem — they would both answer to the same address, and neither the master nor the bus itself can tell them apart.
- Most modules expose solder-bridge address bits, typically 3 of them, good for 8 distinct addresses on otherwise-identical boards — this is the first thing to check before anything else.
- An I2C multiplexer chip, such as the TCA9548A, solves it in hardware instead by switching which physical bus a given address is read from.

Bit-banging I2C in software on plain GPIO pins is technically possible as a last resort, but it means reproducing the entire transaction structure, timing, and arbitration covered above by hand.

Sharing One Address

- A 16×2 character LCD normally needs most of its pins wired directly to the Arduino; a PCF8574-based I2C backpack chip puts the entire parallel interface behind the same two wires everything else on the bus already uses.
- The LiquidCrystal_I2C library wraps the Wire calls entirely — init() and backlight() replace manual pin wiring in setup(), and the rest of the sketch prints to the display exactly like any other LiquidCrystal example.
- The address, 0x27 in **LiquidCrystal_I2C MyLCD(0x27, 16, 2);**, is just another address on the same bus — nothing about talking to a display is different from talking to any other I2C device.

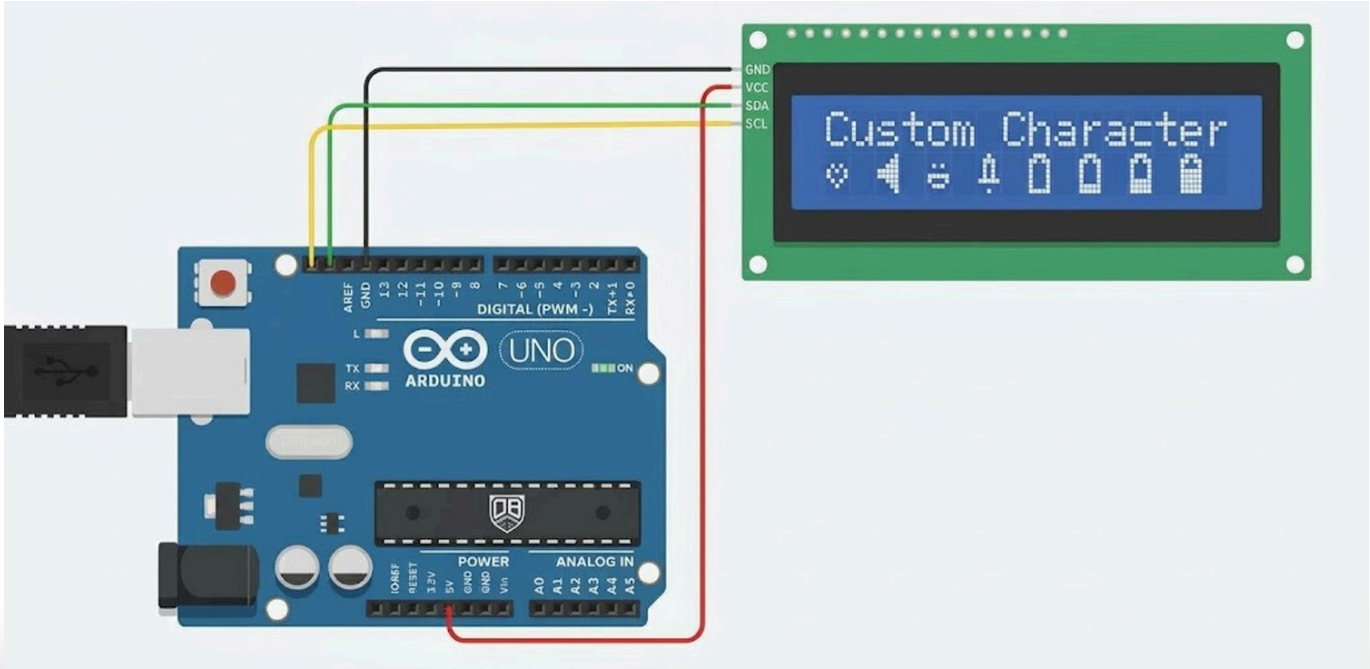
A single extra chip is the entire difference between a display that needs six or more dedicated pins and one that needs exactly two, shared with everything else already on the bus.

A Practical Example: I2C LCD

- init() and backlight() replace the pinMode() calls and direct wiring a plain LiquidCrystal sketch would otherwise need — everything after that is the same display functions either version uses.
- LiquidCrystal_I2C lcd(0x27, 16, 2) sets the address, the column count, and the row count in one line; only those last two numbers change for a different display size.
- setCursor(column, row) followed by print() behaves identically to any other LiquidCrystal display — row 1 here just holds a counter that increments once a second.

```ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup()
{
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Counter:");
}

void loop()
{
  static int count = 0;
  lcd.setCursor(0, 1);
  lcd.print(count);
  count++;
  delay(1000);
}
```

Code: I2C LCD



Four wires total — GND, VCC, SDA, SCL — is everything this display needs, the same two signal wires as any other device already on the bus.

I2C LCD, Wired Up